UNIT-2

Prepared By- Ms. ANJLI
Assistant Professor

Dynamic Memory Allocation

- Dynamic Memory Allocation (DMA) allows a program to allocate memory at **runtime** instead of compile-time. This is useful when we don't know the exact amount of memory needed in advance (e.g., working with **linked lists, trees, graphs, and dynamic arrays**).
- 📌 **Key Benefits:**
  ✔️ Memory is allocated when required and freed when no longer needed.
  ✔️ More efficient memory management.

- There are 4 library functions provided by C defined under **<stdlib.h>** header file to facilitate dynamic memory allocation in C programming. They are:

1. malloc()
2. calloc()
3. free()
4. realloc()

malloc()- Memory Allocation

- Allocates a block of memory but **does not initialize it**. The memory contains garbage values.
- Syntax-

  ptr = (cast_type*) malloc(size_in_bytes);
- Example

  For Example:

ptr = (int*) malloc(100 * sizeof(int));

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.

Malloc()

4 bytes

```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ));
```

A large 20 bytes memory block is dynamically allocated to ptr

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int* ptr;
    int n, i;

    printf("Enter number of elements:");
    scanf("%d",&n);
    printf("Entered number of elements: %d\n", n);

    // Dynamically allocate memory using malloc()
    ptr = (int*)malloc(n * sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        // Memory has been successfully allocated
        printf("Memory successfully allocated using malloc.\n");
```

```c
    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements of the array
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}

return 0;
}
```

```
Enter number of elements:4
Entered number of elements: 4
Memory successfully allocated using malloc.
The elements of the array are: 1, 2, 3, 4,

...Program finished with exit code 0
Press ENTER to exit console.
```

Calloc()- Contiguous allocation

“calloc” or **“contiguous allocation”** method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but It initializes each block with a default value ‘0’.

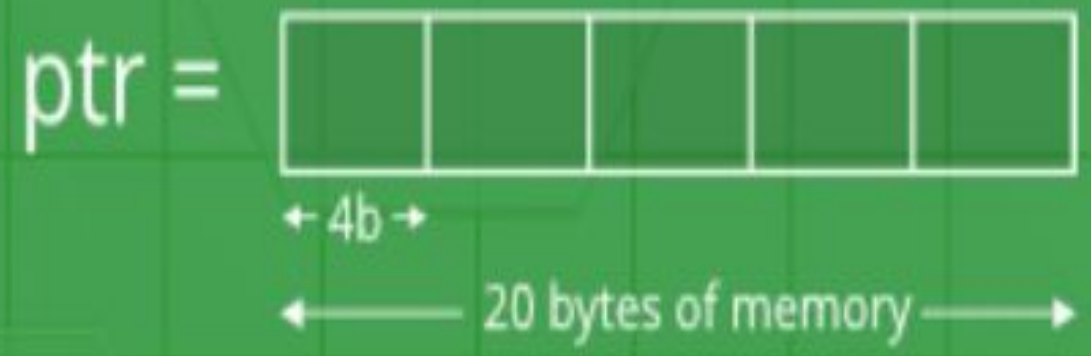
- Syntax-
  ```
  ptr = (cast-type*)calloc(n, element-size);
  ```

- ***Example***
  ptr = (float*) calloc(25, sizeof(float));
  This statement allocates contiguous space in memory for 25 elements each with the size of the float.

Calloc()

4 bytes

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ));
```



5 blocks of 4 bytes each is dynamically allocated to ptr

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int* ptr;
    int n, i;

    // Get the number of elements for the array
    n = 5;
    printf("Number of elements: %d\n", n);

    ptr = (int*)calloc(n, sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        printf("Memory successfully allocated using calloc.\n");

        // Get the elements of the array
        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }
```

```
        printf("The elements of the array are: ");
        for (i = 0; i < n; ++i) {
            printf("%d, ", ptr[i]);
        }
    }

    return 0;
}
```

```
Number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,

...Program finished with exit code 0
Press ENTER to exit console.
```

free()

- **“free”** method in C is used to dynamically **de-allocate** the memory.
- The memory allocated using functions malloc() and calloc() is not de-allocated on their own.
- Hence the free() method is used, whenever the dynamic memory allocation takes place.
- It helps to reduce wastage of memory by freeing it.
- Syntax-

```
free(ptr);
```

Free()

4 bytes

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ));
```

ptr =

5 blocks of 4 bytes each is dynamically allocated to ptr

operation on ptr

free(ptr)

The memory of ptr is released

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int *ptr, *ptr1;
    int n, i;

    n = 5;
    printf("Enter number of elements: %d\n", n);

    ptr = (int*)malloc(n * sizeof(int));

    ptr1 = (int*)calloc(n, sizeof(int));

    if (ptr == NULL || ptr1 == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
```

```
    else {

        printf("Memory successfully allocated using malloc.\n");

        free(ptr);
        printf("Malloc Memory successfully freed.\n");

        printf("\nMemory successfully allocated using calloc.\n");

        free(ptr1);
        printf("Calloc Memory successfully freed.\n");
    }

    return 0;
}
```

```
Enter number of elements: 5
Memory successfully allocated using malloc.
Malloc Memory successfully freed.

Memory successfully allocated using calloc.
Calloc Memory successfully freed.


...Program finished with exit code 0
Press ENTER to exit console.
```

realloc()

- **“realloc”** or **“re-allocation”** method in C is used to dynamically change the memory allocation of a previously allocated memory.
- In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**.
- re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int* ptr;
    int n, i;

    n = 5;
    printf("Enter number of elements: %d\n", n);

    ptr = (int*)calloc(n, sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        printf("Memory successfully allocated using calloc.\n");

        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }
```

```c
printf("The elements of the array are: ");
for (i = 0; i < n; ++i) {
    printf("%d, ", ptr[i]);
}

n = 10;
printf("\n\nEnter the new size of the array: %d\n", n);

// Dynamically re-allocate memory using realloc()
ptr = (int*)realloc(ptr, n * sizeof(int));

  if (ptr == NULL) {
  printf("Reallocation Failed\n");
  exit(0);
}

printf("Memory successfully re-allocated using realloc.\n");

// Get the new elements of the array
for (i = 5; i < n; ++i) {
    ptr[i] = i + 1;
}
```

```
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }

    free(ptr);
}

return 0;
}
```

```
Enter number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,

Enter the new size of the array: 10
Memory successfully re-allocated using realloc.
The elements of the array are: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,

...Program finished with exit code 0
Press ENTER to exit console.
```

S.No	Static Memory Allocation	Dynamic Memory Allocation
1	In the static memory allocation, variables get allocated permanently, till the program executes or function call finishes.	In the Dynamic memory allocation, the memory is controlled by the programmer. It gets allocated whenever a malloc() is executed gets deallocated wherever the free() is executed.
2	Static Memory Allocation is done before program execution.	Dynamic Memory Allocation is done during program execution.
3	It uses stack for managing the static allocation of memory	It uses heap (not heap data structure) of memory for managing the dynamic allocation of memory
4	It is less efficient	It is more efficient
5	In Static Memory Allocation, there is no memory re-usability	In Dynamic Memory Allocation, there is memory re-usability and memory can be freed when not required
6	In static memory allocation, once the memory is allocated, the memory size can not change.	In dynamic memory allocation, when memory is allocated the memory size can be changed.

7	In this memory allocation scheme, we cannot reuse the unused memory.	This allows reusing the memory. The user can allocate more memory when required. Also, the user can release the memory when the user needs it.
8	In this memory allocation scheme, execution is faster than dynamic memory allocation.	In this memory allocation scheme, execution is slower than static memory allocation.
9	In this memory is allocated at compile time.	In this memory is allocated at run time.
10	In this allocated memory remains from start to end of the program.	In this allocated memory can be released at any time during the program.
11	**Example:** This static memory allocation is generally used for array.	**Example:** This dynamic memory allocation is generally used for linked list.